

Intent IDE

Edit as you read. Not after.

Version 6.0 March 2026 Status: Build Spec

Platform: Web app (full surface) → Extension (capture layer)

SECTION 01

Thesis

PRODUCT THESIS

AI is getting good enough to one-shot most documents. The bottleneck is now: how do you review, adjust, and finalize what AI gives you — without re-generating everything you were already happy with?

The current model is insane. AI generates 2,000 words. You read all 2,000. You have thoughts about maybe 200 of them. You write a follow-up. AI regenerates all 2,000 words — burning tokens to reproduce the 1,800 you were fine with, while trying to fix the 200 you weren't. Then you read all 2,000 *again* to figure out what changed. Half the time it changed things you liked.

Intent IDE is a review interface. You read a document. You edit as you go — on the section you're looking at, in the moment. You talk through your thoughts because typing is slow. Every annotation can dispatch an agent if you need it to. Everything you learn, fix, question, and think accumulates across the session — each annotation is informed by everything that came before it. The document updates incrementally. Only what needs to change changes. When you're done reading, you're done.

SECTION 02

Foundations

Non-negotiable. If a feature violates any of these, it does not ship.

01

Edit As You Read

You do not read the whole document then give feedback. You edit each section as you hit it. When you reach the end, the document is done. Reading IS editing. But this is not strictly linear — if you're on paragraph 8 and realize paragraph 2 needs to change, you scroll up, annotate it, and scroll back. The product supports retroactive annotations at any point in the document.

02

Voice-First, Context-Aware

Typing degrades thinking. The primary capture method is voice. When you talk, the system uses your scroll position, cursor position, and last interaction to infer what you're referring to. No eye tracking — the privacy and ethical concerns outweigh the benefit. Cursor and scroll heuristics cover the majority of cases. When ambiguous, the system asks.

03

Every Annotation Can Be Smart

Every annotation type — including thoughts, corrections, and questions — can dispatch a sub-agent. A parked thought can trigger research. A simple correction can ask the agent to verify the fix. A question can cascade into a deep dive. Nothing is "just a note." Everything has the potential to become exploration, and every piece of exploration feeds into the session's accumulating understanding.

04

Scope-Locked Changes

When something changes, only the declared scope changes. The rest is untouched. Enforced architecturally. If you're happy with 1,800 out of 2,000 words, those 1,800 never get re-generated.

05

The Read-Line

Your reading position is a checkpoint. Everything above it is "checked." Changes above the read-line get flagged. Changes below are applied silently. Retroactive annotations (going back up to fix something) are explicitly supported and flagged appropriately.

06

Everything Accumulates

Annotations don't exist in isolation. What you learned in annotation 3 informs the sub-agent resolving annotation 7. What you fixed in section 1 is context for a question in section 4. The system carries forward a growing understanding of your intent, preferences, and decisions across the entire session. By the end of the review, the system knows how you think about this document.

07

Version Control, Not Chat

The underlying model is version control. Changes are tracked individually, reversible individually, with a clear history. But the language is accessible — not git jargon. No one needs to learn "squash" or "cherry-pick" to use this product.

SECTION 03

The Problem

What's broken

Every AI chat interface assumes: (1) read the whole response, (2) write one follow-up referencing everything, (3) wait for full regeneration, (4) re-read everything to verify. Repeat until you give up.

Four failures:

- **No reacting in the moment.** Thought hits on paragraph three. You're supposed to hold it until the reply box. By then it's half-gone.
- **Everything regenerates.** Wanted one sentence changed. Got the whole section rewritten. Now re-read everything to find what else moved.
- **No "what I already read" awareness.** The AI zeroes out your reading investment every time it regenerates.
- **Annotations flattened into messages.** Six thoughts about six sections become one paragraph. Structure gone. Context gone.

Who feels this

Anyone using AI for content longer than a few sentences who cares about quality. PMs writing PRDs. Founders refining decks. Professionals drafting emails. Researchers reviewing reports. Anyone who's said "I just wanted

that part changed."

The deeper insight

AI is approaching one-shot quality. The model often nails it on the first try. The problem is: it wrote something 90% right and you need to fix the 10% without destroying the 90%. That's a review problem. No product solves it.

SECTION 04

How It Works

Step 1: You have a document

Paste it, generate it, import it. It appears in a clean reading surface. This is your Source. Mutable — you can type into it at any time.

Step 2: You start reading

Top to bottom. The read-line tracks your position — the furthest point you've deliberately read (scroll + dwell time).

Step 3: A thought happens

You're on paragraph four. Something catches you. You do one of:

- **Highlight + voice.** Select text, hotkey, talk. System records, anchored to selection.
- **Just voice.** No highlight. Press hotkey, talk. System infers anchor from cursor/scroll position.
- **Direct edit.** Click into the text and type.

Step 4: The system handles it

Annotation captured. Auto-classified. If the type warrants a sub-agent (and almost all of them do), it's dispatched. You keep reading. Dot appears

when result is ready.

Step 5: You check results when you want

Click an annotation. Resolution appears inline. Actions depend on the annotation type (see Section 05).

Step 6: You scroll back up

You're on paragraph 8 and realize something in paragraph 2 needs to change. You scroll up, annotate paragraph 2, scroll back. The system handles this as a retroactive annotation — it's above the read-line, so the change is flagged immediately for your awareness. You already "checked" this section, now something's different. The flag reminds you to re-read what changed.

Step 7: You reach the end

Everything has been addressed as you went. Changes applied incrementally. Diff view shows what changed and why. Done reading = done.

SECTION 05

Annotation Types

Every annotation type can dispatch a sub-agent. The difference is what the agent does and what resolution actions you get. Nothing is "just a note" — everything has the option to go smart.

TYPE	YOU'RE SAYING	WHAT THE AGENT DOES	RESOLUTION ACTIONS
? Question	"I don't get this" / "Is this right?"	Explains, fact-checks, provides context. Does not propose a doc	Got it Go deeper

	/ "Why did you say this?"	change unless you ask it to.	Change this based on what I learned
 Fix	"Change this to X" / "This should say Y"	Proposes scoped replacement following your direction.	Apply Tweak it Nevermind
 Explore	"Tell me more" / "What's the data?" / "Go deeper"	Researches, expands, provides evidence. Presents findings without changing the doc.	Got it Add to doc Keep digging
 Thought	"Hmm, interesting" / "Come back to this" / "I'm not sure yet"	Agent researches the thought — looks up relevant context, checks if the thought has implications for other parts of the doc, surfaces related information. This is not a dead note. It's a seed that the system can grow.	Keep it Act on this Research more Dismiss
 Correction	"The number is 4.2M not 3.8M" / "Wrong spelling"	Agent verifies the correction is accurate (fact-checks the user's proposed change), then applies it. If the correction creates inconsistencies elsewhere in the doc (e.g., a number is referenced in three	Apply Show me what else this affects

		places), the agent flags all of them.	
 Restructure	"This section should flow differently" / "Flip these paragraphs"	Proposes a reorganized version of the scoped section.	Apply Tweak it Nevermind

WHY EVERY TYPE GETS AN AGENT

The v5 spec said Thoughts dispatch no sub-agent and Corrections are direct replacements. That's leaving intelligence on the table. A thought like "hmm, this pricing feels off" could trigger research into competitor pricing that changes how you review the rest of the doc. A correction like "it's 4.2M not 3.8M" should check whether that number appears elsewhere and whether the change breaks any downstream argument. Making every annotation agent-capable means every interaction has the potential to surface something useful. The user doesn't have to think about whether their annotation "deserves" an agent — the system decides based on what would be helpful.

Auto-classification

The system classifies annotation type from voice content or typed text. Uses keyword patterns in v1 (rule-based), upgradeable to a small classifier model. The classified type is shown on the annotation — user can override with one tap. Classification is never exposed during capture. You talk, the system figures it out.

Custom types

Users create custom annotation types in settings: a label, agent

instructions, and resolution actions. Examples: "Flag for Legal" (agent checks for compliance issues), "Needs Data" (agent searches for supporting data), "Too Long" (agent compresses). Created in settings, never during capture.

SECTION 06

Accumulating Context

This is what makes the review experience get better the more you use it within a session. Every annotation and its resolution feeds into the next one.

How it works

The system maintains a session context — a running summary of what the user has done so far:

- **What you've learned:** Questions you asked and their answers. "User asked about the pricing model and was told it's based on X." This informs future sub-agents — if you ask about pricing again in a different section, the agent already knows what you were told.
- **What you've fixed:** Changes you've made. "User changed revenue figure from 3.8M to 4.2M." If a sub-agent encounters the old figure elsewhere, it knows the user already corrected it.
- **What you think:** Parked thoughts and their research results. "User had a thought about competitive positioning — research showed that competitor X launched a similar feature." This shapes how the agent handles related topics later.
- **Your preferences:** Patterns in your edits. If you've consistently shortened verbose paragraphs, the agent learns to be concise. If you've repeatedly asked for data to support claims, the agent starts proactively including evidence.

Context window management

The session context grows throughout the review. To keep sub-agent calls efficient, the system maintains a compressed running summary (~500 tokens) that captures the key decisions, learnings, and preferences. This summary is included in every sub-agent call alongside the annotation-specific context. The full uncompressed history is available on demand.

WHY THIS MATTERS

By annotation 15, the system understands your intent better than it did at annotation 1. It knows your style preferences, the corrections you've made, the topics you care about, and the direction you're pushing the document. Sub-agent responses get sharper as the session progresses because the system is learning from your review in real time. This is fundamentally different from a chat interface where every regeneration starts from scratch.

SECTION 07

Retroactive Annotations

You're on paragraph 8. You realize paragraph 2 is wrong. Maybe you learned something that changes how paragraph 2 should read. Maybe you just remembered something. This needs to work smoothly.

How it works

1. **Scroll up.** The read-line does not move backward — it stays at paragraph 8. This is your high-water mark.
2. **Annotate.** Highlight text in paragraph 2 and voice your annotation. This is captured like any other annotation. The system notes that this is above the read-line — it's a retroactive annotation on "checked" content.

3. **Resolution.** Sub-agent resolves it using the full session context, including everything you've done between paragraphs 2 and 8. This is important: the agent knows what you know now, not just what you knew when you first read paragraph 2.
4. **Apply.** If you accept the change, it modifies paragraph 2. The read-line flag system marks the change immediately — but since you're the one who made it, the flag is auto-acknowledged. You made the change, you know what happened.
5. **Cascade check.** The system checks: does this change to paragraph 2 affect anything in paragraphs 3–7 (above the read-line but between the change and your current position)? If so, those sections get flagged for re-review.
6. **Scroll back.** Return to paragraph 8. Continue reading. Any flags on paragraphs 3–7 are visible in the margin — you can address them now or later.

THE CASCADE PROBLEM

If you change paragraph 2 and paragraphs 3, 5, and 6 reference the same data, all three get flagged. The system should offer: "This change may affect 3 other sections. Want to review them?" with one-click navigation to each flagged section. The user can review or dismiss.

SECTION 08

Focus Inference (No Eye Tracking)

When the user talks without selecting text, the system needs to know what they're referring to.

Anchor inference — priority stack

1. **Cursor position** — if the text cursor is in or near a paragraph, that's the anchor. Highest confidence.
2. **Viewport center** — the content block nearest the center of the visible area. Medium confidence.
3. **Last interaction** — the most recent block the user selected, clicked, or dwelled on. Fallback.

The inferred anchor is shown immediately as a subtle highlight when voice capture begins. If wrong, the user drags it to the right place. Should rarely be needed.

Multi-section references

If the user says "the argument in section two contradicts section four," the system detects section references in the transcript and highlights both. The sub-agent receives context from both.

ON EYE TRACKING

Eye tracking was considered and removed. The privacy and ethical concerns are serious: webcam-based gaze tracking means the product is watching the user's face. Even if the data never leaves the device, the perception is invasive. The consent burden is high, the benefit is incremental over cursor/scroll heuristics, and the user trust cost is significant. This is not a v1 feature and may never be. If it's ever revisited, it requires explicit opt-in, full transparency about what data is captured, and a strong case that cursor heuristics are genuinely insufficient. The product should work well without it.

SECTION 09

The Read-Line

What it is

A soft visual marker showing the furthest point you've deliberately read. Tracked via scroll + dwell time. Does not advance on fast scrolling. Does not move backward.

Behavior

- **Change above the read-line (from a sub-agent):** Flagged. Margin indicator. "Something you already read was modified."
- **Change above the read-line (from your own retroactive annotation):** Flagged, but auto-acknowledged for the specific annotation you made. Cascade effects on other sections are still flagged.
- **Change below the read-line:** Applied silently. You'll encounter the updated version when you get there.

Dwell threshold

Starting default: estimated reading time per block (word count ÷ 250 words per minute) with a minimum of 2 seconds. A 100-word paragraph requires ~24 seconds of dwell to be marked "read." A 20-word heading requires 2 seconds. This is more accurate than a fixed threshold because it scales with content density. Configurable.

SECTION 10

Version Control

The data model is version control. But the language is accessible. No one needs to learn git to use this product.

Concepts — user-facing names

WHAT IT	USER SEES IT AS	UNDER THE HOOD
---------	-----------------	----------------

DOES		
Track a change	"Change" — every edit (from agent or manual) is a tracked change with a reason	Commit with diff, timestamp, and annotation reference
See what changed	"Changes view" — a clear diff showing original vs. current, color-coded by source	Diff between any two versions
Undo a specific change	"Undo this change" — revert one specific edit without affecting others	Revert a specific commit
Dig deeper in multiple rounds	"Thread" — a multi-round conversation on one annotation	Branch (series of commits on one annotation)
Collapse a thread	"Simplify" — collapse a multi-round thread into its final result	Squash merge
Pick parts of a suggestion	"Pick and choose" — accept some parts of an agent's suggestion, reject others	Cherry-pick
Compare versions	"Compare" — side-by-side view of any two versions of the doc	Diff between arbitrary commits
Finish the review	"Finalize" — mark the current version as the final output. Previous versions are saved.	Merge to main, tag release

ACCESSIBLE BY DEFAULT, POWERFUL ON DEMAND

The average user sees "Changes view," "Undo this change," and "Pick and choose." Simple. Intuitive. No learning curve. Power users who know git concepts will recognize the underlying model and

can access advanced features. But the product never requires git knowledge. The cognitive load of "squash" and "cherry-pick" is replaced with plain language that anyone understands.

SECTION 11

Smart Context

A highlighted phrase is often one piece of a multi-section argument. The sub-agent needs to understand the argument, not just the paragraph.

Context package — priority order

1. **The selection**
2. **The local block**
3. **Session context** — the compressed running summary of everything the user has done, learned, and decided so far (from Section 06)
4. **Semantic references** — other document blocks referencing similar concepts
5. **Argument chain** — trace premises to conclusions or conclusions to premises
6. **Prior annotations on related text** — including their resolutions
7. **Generation context** — if the doc was generated in Intent IDE, the original prompt and instructions

Budget

~1,500 tokens of annotation-specific context + ~500 tokens of session context = ~2,000 token context window per sub-agent call. Ranked by priority. Inspectable by the user (expandable panel, hidden by default).

Scope System

LEVEL	TOUCH SURFACE	INFERRED WHEN
phrase	Only the highlighted words	Selection < 1 sentence
sentence	The full sentence	Selection = 1 sentence
paragraph	The full paragraph	Selection spans sentences in one paragraph
section	From heading to heading	Selection spans paragraphs or includes a heading

Auto-inferred. User can adjust with a drag handle. No document-level scope on individual annotations.

Enforcement

Sub-agent output validated against scope boundary. Out-of-scope text is stripped. User notified: "The agent suggested changes beyond your selection." Option to expand scope or keep it tight. This is architectural, not prompt-based — the system enforces it regardless of what the agent returns.

Distribution and Surface

The hard truth

People use Claude and ChatGPT. Getting them to copy-paste into

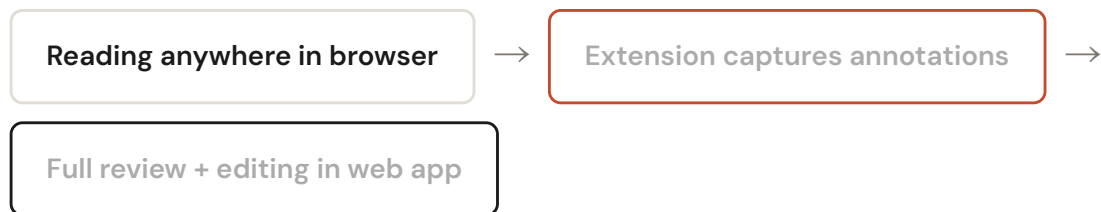
another website is friction. "Another tab" is not a pitch.

Web app is the full surface

This is where the complete experience lives — voice, annotations, inline resolutions, direct editing, changes view, the version control model, everything. A browser extension sidebar is too small for real editing. The web app is the workshop.

Extension is the capture layer

When you're reading in Claude, ChatGPT, Google Docs, or any browser page — the extension lets you highlight and voice-annotate right there. Annotations sync to the web app. But actual review, editing, and changes — that happens in the full app.



BYOK (Bring Your Own Key)

Users bring their own Claude/OpenAI key. Intent IDE charges for the review interface, not the AI. The AI is commodity. The review process is the value.

EXISTENTIAL RISK

If Anthropic or OpenAI builds inline annotation natively, a standalone product loses its hook. Defense: (1) ship first, (2) works across all AI tools, (3) voice-first + read-line + version control + accumulating context are deeper than what they'd bolt onto chat, (4) BYOK means model-agnostic.

Non-Goals

- **Real-time collaboration.** v1 is single-player.
- **Document generation as the primary feature.** Generation is a convenience. The product is the review.
- **Full rich-text editor.** Basic formatting. Not Google Docs.
- **Offline mode.**
- **Mobile-native app.**
- **Eye tracking.** Ethical concerns outweigh benefit. See Section 08.
- **Graph / thinking visualization.** May return as analytics. Not core.

Core Objects

OBJECT	DEFINITION
Source	The document. Mutable. Versioned. Every state preserved.
Annotation	User reaction anchored to text. Voice transcript or typed note. Auto-classified type. Scope. Context package. Resolution thread.
Resolution	Sub-agent response. May be informational or a proposed change. Informed by session context.
Change	A tracked modification. From accepted resolution, manual edit, or correction. Has diff, scope, timestamp, annotation link.
Read-Line	Cognitive checkpoint. Tracked by scroll + estimated reading time dwell.

Session Context	Running summary of user's learnings, fixes, preferences, and thoughts. Compressed to ~500 tokens. Fed to every sub-agent.
Session	A review pass. All annotations, resolutions, changes, read-line history, session context evolution. Produces a new Source version when finalized.

SECTION 16

Technical Architecture

Sub-agent prompt

```
SYSTEM: You are a scoped review agent.

SESSION CONTEXT (what the user has done so far):
{session_context_summary}

ANNOTATION:
  Type: {type}
  Scope: {scope}
  Selection: "{selection}"
  User said: "{transcript}"

CONTEXT:
  Local block: "{local_block}"
  Semantic refs: "{semantic_refs}"
  Argument chain: "{argument_chain}"
  Prior annotations: "{prior_annotations}"
  Generation context: "{generation_context}"

RULES:
1. You are informed by the session context. The user's
   prior decisions, learnings, and preferences should
   shape your response.
2. For QUESTION: explain. Only propose changes if asked.
3. For FIX/RESTRUCTURE: return replacement within scope.
```

4. For EXPLORE: provide info. Don't auto-change doc.
5. For THOUGHT: research the thought. Surface relevant info, implications, related parts of the doc. This is not a dead note – help the user develop the thought.
6. For CORRECTION: verify accuracy. Check for downstream references. Apply if correct, flag inconsistencies.
7. NEVER modify outside declared scope boundary.

Session context summarizer

After each annotation resolution, the session context is updated. A lightweight summarization pass compresses the growing history into ~500 tokens. This runs client-side or via a fast API call. The summary captures: decisions made, corrections applied, preferences detected, questions answered, thoughts parked. This summary is the "accumulated understanding" that makes each successive sub-agent call smarter.

Voice pipeline

STEP	TECH	LATENCY
Record	MediaRecorder API	< 100ms to start
Transcribe	Whisper API (streaming) + Web Speech API fallback	< 2s after stop
Classify	Small classifier model (not limited to keyword rules)	< 500ms
Anchor	Selection or focus heuristic	Instant
Context	Embedding similarity + session context	< 1s
Sub-agent	Claude API (streaming)	First token < 2s

Stack

LAYER	TECHNOLOGY
Frontend	React + TypeScript
Text engine	ProseMirror (annotation anchoring, direct editing, transactional changes)
Voice	Whisper API + Web Speech API fallback
Classifier	Small fine-tuned model or distilled classifier (not just keyword rules)
LLM	Claude API (default), OpenAI-compatible for BYOK
Context engine	Transformers.js (client-side embeddings) + session summarizer
Version control	Custom change log over ProseMirror transactions
Backend	Next.js API routes
Storage	PostgreSQL + Redis
Auth	Clerk
Extension	Chrome Extension Manifest V3

SECTION 17

Business Model

TIER	PRICE	WHAT YOU GET
------	-------	--------------

Free	\$0	3 sessions/month, 10 annotations/session. Our API.
Pro (BYOK)	\$12/month	Unlimited. Your API key. Voice. Extension. Export. Custom types. Session history. Changes view.
Pro (Included AI)	\$25/month	Everything, we handle the AI.
Team	\$35/user/month	Shared sessions. Team analytics. SSO.

Token efficiency as selling point

Scoped sub-agents: ~500 tokens per call (including session context). 10 fixes in a 2,000-word doc = ~5,000 tokens. Regenerating the full doc 10 times = ~40,000+ tokens. Intent IDE is 8x more efficient. For BYOK users, this directly reduces their API bill.

SECTION 18

Success Metrics

METRIC	TYPE	TARGET	WHEN
Highlight → voice recording starts	Primary	< 1 second	Prototype
Voice stops → annotation captured	Primary	< 2 seconds	Prototype
Focus anchor accuracy	Primary	≥ 80%	Prototype
Auto-classification	Primary	≥ 80%	Prototype

accuracy			
Annotations per session	Primary	≥ 5	90 days
Annotation type distribution	Tracking	All types used (no single > 60%)	90 days
Resolution usefulness	Primary	≥ 60% acted on	90 days
Thought → action conversion	Secondary	≥ 30% of Thoughts lead to doc changes	90 days
Session context improvement (resolution quality improves from annotation 1 to 10)	Secondary	Measurable improvement in acceptance rate late vs. early in session	90 days
Retroactive annotations per session	Tracking	Track frequency and cascade engagement	90 days
Scope violations	Guardrail	< 5%	Ongoing
WAU	Growth	1,000	6 months

SECTION 19

Roadmap

PHASE	WHAT SHIPS	PROVES
0	Core loop prototype. Paste-in source. Voice capture. Auto-classification (small	Voice natural? Focus accurate?

	classifier, not just keywords). Focus anchoring. Inline resolutions with type-appropriate actions. Direct editing. Changes view. No auth, no persistence.	Classifier accurate? Do people edit as they read?
1	Web app MVP. Auth, sessions, read-line, cascading flags, retroactive annotations, accumulating session context, version control with accessible naming, BYOK, custom types, export.	Do people come back? Does session context improve resolution quality? Does the read-line add value?
2	Browser extension (capture layer). Annotate on any page. Sync to web app.	Do people capture outside the app? Does capture → app flow work?
3	Smart context engine. Argument chain tracing, semantic references, cross-annotation threading, generation context awareness.	Does smarter context improve resolution quality? Measured by acceptance rate delta.

SECTION 20

Risks

RISK	SEVERITY	MITIGATION
Voice capture awkward or	High	Whisper + browser fallback. Typing always available. Test with 10 people

unreliable		first.
Focus anchoring wrong	High	Show anchor immediately. One-drag correction. Log accuracy. No eye tracking — privacy risk too high.
People won't go to another website	High	Extension meets users where they read. BYOK prevents subscription fatigue. Web app is the workspace, not the entry point.
Auto-classification wrong often	Medium	Small classifier model from day one (not just keywords). One-tap override. Log overrides to improve model.
Session context gets noisy	Medium	Aggressive compression. Prioritize decisions and corrections over exploratory noise. Tune summarizer with real session data.
Retroactive annotations create too many cascade flags	Medium	Only flag sections with direct references to changed content (semantic match, not positional). Batch flags into one notification.
Anthropic/OpenAI builds this natively	Medium	Speed. Cross-platform. Voice + read-line + accumulating context + version control = deeper than chat bolt-on.
Version control concepts confuse non-technical users	Medium	Accessible naming (Section 10). "Undo this change" not "revert." "Pick and choose" not "cherry-pick." Never expose git jargon.

Open Questions

QUESTION	NEEDED BY
What's the best small classifier model for annotation type classification? Fine-tuned distilBERT? Something smaller that runs client-side?	Phase 0
Web Speech API vs. Whisper vs. hybrid? Accuracy, latency, cost tradeoffs.	Phase 0
ProseMirror vs. TipTap vs. Lexical?	Phase 0
How to compress session context effectively? Summarization model? Template-based extraction?	Phase 1
Dwell threshold: reading-time-based (word count ÷ 250 wpm) or fixed? How to handle scanning vs. reading?	Phase 1
Extension CSP restrictions on Claude.ai, ChatGPT, Google Docs?	Phase 2
Provisional patent: voice-capture + focus inference + scoped agents + read-line + accumulating context + version control. Patentable?	Before launch

SECTION 22

Next Steps

1. **Register intentide.com**
2. **Build Phase 0 prototype** — paste a doc, voice annotate, auto-classify (small model, not keywords), focus anchoring, inline

resolutions with type-appropriate actions, direct editing, changes view. No auth.

3. **Test three things:** (1) voice capture feel, (2) focus anchor accuracy, (3) classifier accuracy. 10 people. Stopwatch.
4. **3-minute Loom demo**
5. **Provisional patent**
6. **Landing page + waitlist**

THE TEST

Put the prototype in front of someone. Give them a 1,000-word AI-generated PRD. Say: "Read this and react however you want." If they naturally talk through their thoughts, edit as they go, and reach the end with a document they're happy with — the product works. If at any point they stop and say "how do I..." — the interaction is broken. Fix it before building anything else.

APPENDIX

Version History

v1.0 — Original concept. Inline annotation + sub-agents + split canvas + synthesis as merge.

v2.0 — Full product spec. Git-like versioning. Objects, flows, architecture.

v3.0 — Five foundations: capture-first, minimum viable annotation, scope-aware, invisible sub-agents, graph as payoff.

v4.0 — Voice-first. Auto-classification. Read-line. Direct editing. Graph demoted. BYOK.

v5.0 — Edit-as-you-read. Annotations as exploration (not binary decisions). Gaze awareness. Git formalized. Extension as capture layer.

Token efficiency as pitch.

v6.0 (this version) — Key changes: every annotation type dispatches an agent (Thoughts get research, Corrections get verification and cascade detection). Retroactive annotations formalized (annotating above read-line with cascade flagging). Accumulating session context — every annotation informs the next, the system learns your intent throughout the review. Git jargon replaced with accessible names ("Undo this change" not "revert," "Pick and choose" not "cherry-pick"). Eye tracking removed on ethical grounds — will not be revisited without strong justification. Classifier upgraded from keyword rules to small model from Phase 0. Dwell threshold changed from fixed time to estimated reading time per block.